Edge Detection

Week 5 — Finding boundaries with gradients

Practical Image Processing

Lecture 5 ▪ 60–70 min

Recap & today

- Wk3–4: smoothing/denoising with kernels.
- Today: **edges** — where intensity changes sharply. We find them with **gradients** (a high-pass kernel), then build up to **Canny**.

DocScan milestone

Detecting the page/meter boundary starts with edges. This week's output feeds the "find the page" step.

Roadmap

What is an edge?

Second derivatives

The Canny edge detector

In-class exercise

Deep dive

Wrap-up

What is an edge?

An edge = a rapid intensity change

- Where the image jumps from dark to light (or vice versa) — object boundaries, text strokes.
- Mathematically, a large **gradient**: the derivative of intensity is high there.
- Edges are **high-frequency** (Week 3) — so we detect them with a high-pass kernel.

Math

Gradient magnitude $|\nabla f| = \sqrt{f_x^2 + f_y^2}$; direction $\theta = \text{atan2}(f_y, f_x)$.

Derivatives as kernels

Sobel-x (vertical edges):

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Sobel-y (horizontal edges):

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Each computes a *difference* across the pixel — a discrete derivative. Sum to 0 (flat regions give 0, as an edge kernel should).

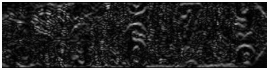
Sobel on the real meter



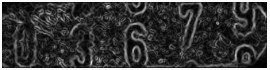
gray



∂x (vertical edges)



∂y (horizontal)



magnitude

x-gradient highlights vertical strokes; y-gradient horizontal ones; magnitude combines both.

Gradient magnitude & direction

- **Magnitude** $= \sqrt{g_x^2 + g_y^2}$: how strong the edge is (the bright edge map).
- **Direction** $= \text{atan2}(g_y, g_x)$: which way the edge points (used by Canny).
- Threshold the magnitude to get a binary edge map — but a fixed threshold is fragile.

✕ Pitfall

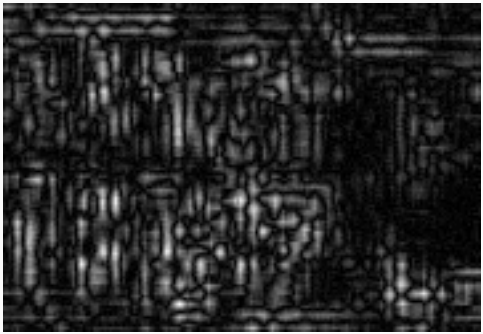
Gradients are noise-sensitive: noise creates strong fake gradients. Always smooth first (Week 3/4).

Second derivatives

Laplacian: the second derivative

$$\text{Laplacian} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

Detects edges as **zero-crossings** (where the 2nd derivative flips sign).



Laplacian of a plate

- One kernel finds edges in all directions — but even more noise-sensitive.
- **LoG** (Laplacian of Gaussian) smooths first, then applies the Laplacian.

The operator family

Operator	Idea	Notes
Roberts	2×2 diagonal diff	tiny, very noise-sensitive
Prewitt	3×3, equal weights	simple gradient
Sobel	3×3, centre-weighted	the practical default
Scharr	optimized 3×3	better rotational accuracy
Laplacian	2nd derivative	all directions, one kernel

Key idea

They all estimate a derivative. Sobel/Scharr are the go-to first-derivative operators.

The Canny edge detector

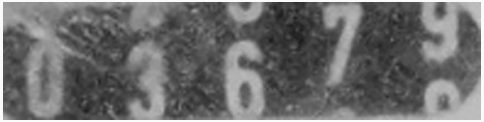
Canny: the gold-standard pipeline

1. **Smooth** with a Gaussian (kill noise).
2. **Gradient** magnitude & direction (Sobel).
3. **Non-maximum suppression**: thin thick edges to 1-pixel ridges.
4. **Double threshold + hysteresis**: keep strong edges, and weak ones *connected* to strong.

Key idea

Canny is not a kernel — it's a 4-step algorithm that produces clean, thin, connected edges.

Canny on real images



meter



Canny edges



plate



Hysteresis: the clever part

- Two thresholds: **high** (definitely an edge) and **low** (maybe).
- Keep every strong edge, plus weak edges that *touch* a strong one — drop isolated weak ones.
- This bridges gaps along a real edge while rejecting scattered noise responses.

✔ Good practice

Hysteresis is why Canny gives long, connected contours instead of a speckled mess.

Canny in code + tuning

```
blur  = cv2.GaussianBlur(gray, (5, 5), 0)      # step 1: smooth
edges = cv2.Canny(blur, 80, 160)               # low=80, high=160
# rule of thumb: high ~= 2-3x low; set from the median intensity (auto-Canny)
```

✕ Pitfall

Skipping the blur (or thresholds too low) floods the image with noise edges — compare `Canny(80,160)` to a low-threshold run.

In-class exercise

In-class exercise (12 min)

Reason about a 1D signal

A row of pixels: `[10, 10, 10, 200, 200, 200]`.

1. Where is the edge? What is the (first) derivative at each gap (differences)?
2. Where is the derivative *maximum*? Relate that to "gradient = edge".
3. If one pixel were noise (`[10,10,90,200,200,200]`), how would a gradient react — and how does smoothing help?

Debrief

- Differences: `[0,0,190,0,0]` — the derivative spikes at the $10 \to 200$ jump. That spike *is* the edge.
- Max derivative = the edge location. Thresholding the derivative finds edges.
- The noise pixel (90) creates a smaller spurious gradient; smoothing averages it away before it becomes a false edge.

Key idea

An edge is where the derivative is large. Everything this week is a way to compute and clean that derivative.

Deep dive

Why smooth before differentiating

- Differentiation **amplifies** high frequencies — including noise.
- So the derivative of a noisy signal is dominated by noise, not edges.
- Smooth first (Gaussian), then differentiate — the basis of Sobel/Canny/LoG.

Math

Handy identity: (blur then differentiate) = convolve with the *derivative of the Gaussian* — one kernel.

Sub-pixel & oriented edges (brief)

- Edge *location* can be refined to sub-pixel accuracy by interpolating the gradient — matters for precise measurement (Week 9).
- The gradient *direction* lets you find edges of a specific orientation (e.g. only vertical strokes).
- Oriented filters (Gabor) respond to edges/texture at a chosen angle and scale.

Choosing thresholds: auto-Canny

- Fixed thresholds fail across different lightings. Derive them from the image's median.
- `low = 0.66*median`, `high = 1.33*median` is a robust starting recipe.
- Then tune on *hard* images (shadowed, low-contrast meter photos).

Edges → the DocScan page finder

- Canny gives the boundary of the page/meter as connected edges.
- Next (Week 9): find **contours** from these edges, keep the largest 4-sided one — the page.
- Then (Week 11): use its 4 corners for perspective correction. This week is step one.

Key idea

Edges aren't the goal — they're the raw material for finding *shapes* (Week 9) and *structure*.

Gradient magnitude vs Canny — what's the difference?

- Thresholded Sobel magnitude: thick, broken, noisy edges.
- Canny: thin (1-pixel), connected, clean — thanks to NMS + hysteresis.
- Use raw gradients for texture/feature strength; use Canny when you need clean *contours*.

✔ Good practice

For finding the page boundary (a contour), Canny. For measuring edge strength, the gradient magnitude.

Edges and deep learning

- A CNN's first layer learns edge/gradient filters *automatically* — often resembling Sobel.
- So the features you compute by hand today are what a network discovers on its own (Week 13).
- Classical edges remain useful: fast, interpretable, no training data needed.

Wrap-up

What you should now be able to do

- Explain edges as large gradients; compute Sobel-x/y, magnitude, direction.
- Use the Laplacian/LoG; understand zero-crossings.
- Run and tune the 4-step Canny detector; know why each step exists.
- Connect edges to the DocScan page-finding pipeline.

This week's deliverable

Due before Week 6 (see `project/`)

1. Compute Sobel-x/y and magnitude on a meter; visualize each.
2. Run Canny; show the effect of smoothing (with vs without) and of the two thresholds.
3. Implement auto-Canny from the median; compare to hand-tuned thresholds.
4. DocScan: get clean edges of the page/meter boundary on 3 real photos.

An edge is where the derivative is large.

Smooth, differentiate, thin, connect — that's Canny.